

Service Oriented Architecture (SOA)

Detailed Structured Notes & Explanation



Part 1: Introduction to SOA



1. Introduction to Service Oriented Architecture (SOA)

- Service Oriented Architecture (SOA) is both a specification and a methodology used to design distributed applications.
- The **main goal** of SOA is to provide platform-independent and language-independent services, so that systems written in different languages and running on different platforms can work together.
- In SOA, a **service** represents a repeatable business task, such as authentication, payment processing, or order tracking.
- A **business process** is created by combining multiple services together in a logical sequence.
- SOA follows a **component-based architecture**, where each component exposes its functionality as a service.
- Communication in SOA happens using a **message-passing model**, meaning services interact by exchanging messages rather than directly calling each other's code.



2. Message-Based Communication in SOA

- In SOA, a client (service consumer) **does not directly access** the internal logic of a service.
- Instead, the client sends a message in a standard format containing:
 - The data to be processed.
 - Metadata describing how the data should be handled.
- The service (provider) processes the message and sends a response message back to the client.
- A common example of a message format is an **XML document**.
- These messages are transported over network protocols such as:
 - SOAP
 - HTTP
 - REST
 - JMS
- The service exposes itself to clients through an **endpoint**, which is the address where the service can be accessed.

3. Role of Middleware in SOA

- In real SOA implementations, service providers and service consumers do not communicate directly.
- Instead, communication is handled by **middleware software**, which acts as:
 - A broker
 - A transaction manager
 - A translator
- Middleware performs the following important functions:
 - Discovers available services.
 - Registers services in a registry.
 - Translates messages between different formats.
 - Routes messages to the correct service.

- Middleware may also contain **business process logic**, meaning complex workflows can be handled centrally.
- Because SOA is a distributed architecture, middleware often includes:
 - Security services
 - Trust services
 - Access control mechanisms

4. Service Discovery and UDDI

- To enable service discovery, SOA uses a **registry** where services are published.
- The most commonly used protocol for this purpose is **UDDI (Universal Description, Discovery, and Integration)**.
- UDDI allows:
 - Service providers to publish services.
 - Service consumers to discover services.
- Service descriptions are often exchanged using **ebXML documents**.
- Once a service is found, the client **binds** to that service and starts sending requests.
- If multiple services are available, the broker can choose the most appropriate one.

5. Loose Coupling and Interface-Based Interaction

- SOA does not use executable links or direct API dependencies.
- The client only knows:
 - The service interface.
 - The message format.
- The client **does not know**:
 - How the service is implemented.
 - Where the service is located.

- Which operating system or programming language is used.
- This makes SOA **loosely coupled**, which increases flexibility.
- The client itself is responsible for validating whether the returned response is correct.



6. Messaging Standards and Protocol Stack in SOA

- The most commonly used message format in SOA is **XML with SOAP**.
- Other important standards used in SOA include:
 - **WSDL** – describes the service interface.
 - **WSS** – provides security features.
 - **WS-BPEL** – used to define business process logic.
- **WSDL (Web Services Description Language)** defines:
 - What the service does.
 - How to access it.
 - What data formats are used.
- **SCDL (Service Component Definition Language)** defines component-level details not covered by WSDL.



7. Service Contract in SOA

- The formal definition of how a service behaves is called a **service contract**.
- A service contract specifies:
 - What the service does.
 - How messages are exchanged.
 - Expected Quality of Service (QoS).
- In commercial systems, contracts may include:
 - Service Level Agreements (SLAs).
 - Performance guarantees.

- Availability requirements.
- The concept of a contract ensures predictable and reliable service behavior.



8. Orchestration and Choreography

ORCHESTRATION

- In orchestration, a **central middleware component** controls the entire business process.
- All services communicate only with the orchestrator.
- The business logic exists in one central place.
- Orchestration makes:
 - Change management easier.
 - Error handling centralized.
 - Monitoring and logging simpler.

CHOREOGRAPHY

- In choreography, there is **no central controller**.
- Each service knows:
 - When to execute.
 - Which service to interact with.
- Business logic is distributed across services.
- Choreography requires higher coordination and is harder to manage.



9. Business Process Execution and Modeling

- **WS-BPEL** is used to define executable business processes.
- It supports:
 - Conditional logic
 - Loops

- Variables
 - Fault handling
- Complex business processes are often modeled using **UML diagrams**, such as:
 - Sequence diagrams
 - Activity diagrams
 - Component diagrams

10. Event-Driven SOA (SOA 2.0)

- SOA 2.0 extends traditional SOA by adding **event handling**.
- Events may:
 - Trigger a business process.
 - Modify an existing process.
 - Stop a process if conditions are not met.
- A **Causal Vector Engine (CVE)** is used to:
 - Analyze event sequences.
 - Detect patterns.
 - Trigger actions using rule-based logic.
- CVEs often include dashboards to monitor events in real time.

11. Virtualization and Black-Box Nature of Services

- From the client's perspective, the service behaves like a **black box**.
- The client does not know:
 - Internal logic.
 - Deployment location.
- Services can be:
 - Updated

- Replaced
 - Migrated
- All without affecting the client.
- This leads to service virtualization and better system scalability.



12. Enterprise Service Bus (ESB)

- An ESB is an architectural pattern used in SOA.
- It acts as a message transaction system, not a physical network bus.
- ESB provides:
 - Message routing
 - Message transformation
 - Logging and auditing
 - Transaction management
- ESB ensures transactional integrity, similar to **ACID properties** in databases.



13. Repository vs Registry

- **Repository:**
 - Stores service implementations.
 - Stores source code and technical artifacts.
 - Acts like a name server.
- **Registry:**
 - Stores metadata about services.
 - Stores service descriptions and policies.
 - Acts like a directory service.



14. Service Catalogs

- Service catalogs help locate services in large SOA systems.
- They store:
 - Available services
 - Dependencies
 - Ownership
 - SLA history
- Catalogs may be:
 - Standalone
 - Global
 - Federated
- Service catalogs improve performance and governance in large internetworks.



15. SOA Communication Protocols

- SOA communication uses:
 - Data interchange formats (XML, SOAP).
 - Network protocols (HTTP, REST, JMS).
- **SOAP** uses XML and relies on application-layer protocols.
- **REST** uses standard HTTP methods like:
 - GET
 - POST
 - PUT
 - DELETE
- REST is lightweight and easy to implement.
- SOAP provides stronger typing and better error handling.

- Modern systems often use both SOAP and REST together.



16. WSDL Structure and Service Contract Expansion

- A WSDL file defines:
 - Service
 - Endpoint
 - Binding
 - PortType
 - Operations
 - Messages
 - Data types
- WSDL alone is not enough.
- SOA expands this through a **service contract**, which includes:
 - Headers
 - Service type
 - Functional specification
 - Transaction attributes
 - Security parameters
- This makes SOA more comprehensive than basic Web services.



Part 2: Business Process Execution Language (BPEL)



1. Need for BPEL in SOA

- In Service Oriented Architecture (SOA), a single message usually represents one **atomic transaction**, meaning a very small unit of work such as a service request or response.
- However, real-world business operations are not completed using a single transaction; they require multiple transactions to work together in a coordinated manner.
- Therefore, the next level of abstraction in SOA is the grouping, sequencing, and management of multiple transactions to form meaningful work.
- This coordinated execution of multiple services is known as a **business process**, and BPEL was created to define and execute such processes.



2. Definition of BPEL

- Business Process Execution Language (BPEL), also known as **Web Services Business Process Execution Language (WS-BPEL)**, is a standard execution language used to define interactions between Web services.
- BPEL is maintained by **OASIS** (Organization for the Advancement of Structured Information Standards) through its WSBPEL Technical Committee.
- BPEL allows developers to describe how multiple Web services interact, in what order they execute, and how data flows between them.



3. BPEL as a Meta-Language

- BPEL is described as a **meta-language**, meaning its statements operate on or control statements written in other languages.
- The languages that BPEL controls are called **object languages**, such as WSDL-defined Web services.
- BPEL itself does not perform business logic directly; instead, it coordinates and orchestrates services that do.



4. Functions of BPEL

- BPEL performs two major functions:

EXECUTABLE COMMANDS

- These commands control interactions between Web services and clients.
- They specify service invocation, message sending, message receiving, and synchronization.

ABSTRACT OR INTERNAL BUSINESS LOGIC

- This part defines how the internal business logic flows.
- It includes conditions, loops, branching, and sequencing of service calls.



5. Role of BPEL in SOA

- BPEL is widely used to compose, orchestrate, and coordinate business processes in an SOA environment.
- **Orchestration** means that BPEL centrally controls which service runs, when it runs, and what data it receives.
- BPEL supports **asynchronous communication**, which is essential because SOA transactions are often long-running.



6. Technologies Used by BPEL

- BPEL is written using **XML**, making it platform-independent and interoperable.
- It integrates with several Web service standards, including:
 - SOAP for message exchange.
 - WSDL for service descriptions.
 - UDDI for service discovery.

- WS-Reliable Messaging for guaranteed delivery.
- WS-Addressing for message routing.
- WS-Coordination and WS-Transactions for transaction control.
- BPEL is built upon earlier technologies:
 - IBM's WSFL (Web Services Flow Language), which uses directed graphs.
 - Microsoft's XLANG, which is block-structured and business-process oriented.
- These were merged to form BPEL4WS, which later evolved into modern BPEL standards.



7. Human Interaction in BPEL

- Traditional BPEL focuses on automated Web services.
- To support human participation in workflows, **BPEL4People** was introduced.
- BPEL4People comes under **WS-HumanTask** specifications by OASIS.
- This allows tasks such as approvals, reviews, and manual decision-making to be included in business processes.



8. BPEL Process Views

- A BPEL business process has two views:
 - Executable (Internal) View.
 - Abstract (External) View.
- The **executable view** defines how the process actually runs.
- The **abstract view** defines how the process interacts with other processes, without revealing internal logic.
- This separation improves reuse and interoperability.

9. Features of BPEL

- BPEL supports process data and control flow, allowing management of variables and execution order.
- It manages **process instances**, meaning multiple executions of the same business process can run simultaneously.
- It supports logic and branching, such as conditional execution and loops.
- Since transactions are long-lived and asynchronous, BPEL provides:
 - Error handling.
 - Scoping of transactions.
 - Compensation mechanisms.
- BPEL relies heavily on Web service standards to assemble and decompose processes.



Part 3: Business Process Modeling in SOA



10. Purpose of Business Process Modeling

- The goal of SOA is not just service access, but the creation of complete business processes.
- In complex systems involving many clients and services, understanding the overall workflow becomes difficult.
- Business process modeling tools help visualize, design, optimize, and manage SOA systems.



11. Importance of Modeling Tools

- Modeling business processes is similar to designing databases using ER diagrams or CASE tools.
- These tools help developers:
 - Understand system structure.
 - Optimize workflows.
 - Manage lifecycle and changes.



Common SOA Modeling Methodologies

12. UNIFIED MODELING LANGUAGE (UML)

- UML is developed by the Object Management Group (OMG).
- It provides graphical diagrams to represent software systems.
- UML models actors, business processes, components, databases, and activities.
- UML includes:
 - Structural diagrams (system components).
 - Behavioral diagrams (states and actions).
- UML models can be automatically converted into code.

13. XML METADATA INTERCHANGE (XMI)

- XMI is also an OMG standard.
- It is used to exchange metadata using XML.
- UML models commonly use XMI for interchange.
- XMI is standardized as ISO/IEC 19503:2005.

14. SYSTEMS MODELING LANGUAGE (SYSML)

- SysML is a simplified extension of UML.
- It reuses 7 UML diagrams and is easier to learn.
- SysML supports XMI and works toward ISO 10303 (STEP) standards.

15. BUSINESS PROCESS MODELING NOTATION (BPMN)

- BPMN represents business processes using Business Process Diagrams (BPDs).
- It visually illustrates workflows similar to UML.
- BPMN diagrams can be converted into WS-BPEL, though conversion is tool-specific.



16. Service-Oriented Modeling Framework (SOMF)

- SOMF was proposed by Michael Bell.
- It provides a graphical map of SOA components and relationships.
- SOMF supports:
 - Optimization
 - Message tracing
 - Architecture planning
 - "What-if" analysis



Part 4: SOMF Concepts



17. SOMA (IBM's SOMF Implementation)

- SOMA (Service-Oriented Modeling and Architecture) was introduced by IBM in 2004.
- It divides systems into:
 - Services
 - Service components
 - Information flows
- SOMA helps determine:
 - Service boundaries

- Service granularity
- Asset reuse



18. SOMF Service Lifecycle

- **Conceptualization** – define supported processes.
- **Discovery** – expose components.
- **Analysis** – identify service-client relationships.
- **Integration** – implement messaging.
- **Logical design** – orchestration/choreography.
- **Architecture design** – define interfaces.
- **Implementation** – build and test.

19. Message-Passing Topologies in SOMF

CIRCULAR TOPOLOGY

- No central controller.
- Each service knows where to send the next message.
- Choreography is decentralized.

HIERARCHICAL TOPOLOGY

- Tree structure with parent-child relationships.
- Central logic at root.
- High overhead in message routing.

NETWORK TOPOLOGY

- Many-to-many connections.
- Fast message passing.
- Complex link management.

STAR TOPOLOGY

- Central service coordinates all communication.
- Used in orchestration and publish-subscribe systems.

20. Types of Services in SOMF

- **Atomic Service**
 - Very small and indivisible.
 - Performs one useful function.
 - Example: customer lookup.
- **Composite Service**
 - Collection of atomic services.
 - Multi-functional and coarse-grained.
 - Example: ESB.
- **Service Cluster**
 - Group of composite services.
 - Forms a complete business module.
 - Example: payroll system.



Part 5: Managing, Security & Cloud



Managing and Monitoring SOA

21. NEED FOR SOA MANAGEMENT

- SOA systems are distributed and virtualized.
- Failures cannot be handled reactively.

- Continuous monitoring and proactive management are required.

22. SOA MANAGEMENT TOOLS

- HP OpenView SOA Manager
- IBM Tivoli ITCAM
- Oracle BPEL Process Manager
- These tools provide:
 - Monitoring
 - Change management
 - Lifecycle management



SOA Security

23. SECURITY CHALLENGES

- SOA transmits large volumes of messages.
- Traditional application-level security is insufficient.

24. SOA SECURITY STANDARDS

- **SAML** – authentication and authorization.
- **WS-Security** – message-level security.
- **WS-SecureConversation** – security context.
- **WS-SecurityPolicy** – policy enforcement.
- **WS-Trust** – token issuance and validation.

25. XML GATEWAY SECURITY

- XML gateways intercept SOAP/REST messages.
- PKI is used for encryption and digital signatures.

SOA and Cloud Computing

26. RELATIONSHIP BETWEEN SOA AND CLOUD

- Cloud computing does not require SOA.
- As cloud applications grow complex, SOA becomes useful.
- SOA provides loose coupling and flexibility.

27. SOA AND WEB 2.0

- Web 2.0 applications often use SOA components.
- Technologies include REST, AJAX, and mashups.
- SOA supports the vision of an Internet of Services.

Conclusion (Exam-Friendly)

- SOA provides a structured way to build distributed systems.
- BPEL enables execution of complex business processes.
- Modeling frameworks like SOMF simplify system design.
- Management and security are critical in SOA deployments.
- SOA and cloud computing complement each other for scalable systems.